

Estructuras de Datos (GPD4402) - Tarea 7: Arbol AVL (Arbol Binario de Búsqueda Balanceado)

Documento de reflexion

Igal Shturman Poplawsky - Matricula 18139

1. Sobre la comparacion: mi BST midio 15 y mi AVL midio 4

En mi demo de la Parte 4 meti los numeros del 1 al 15 en orden creciente en los dos arboles: en mi Tree de la practica 5 y en mi AVLTree nuevo. El BST midio altura 15 y el AVL midio altura 4. El in-orden de los dos salio identico: 1, 2, 3, ..., 15. El BST llega a 15 porque cada numero nuevo es mas grande que todos los anteriores, asi que mi InsertRec siempre baja a la derecha y nunca se acomoda: queda como una lista con pasos extra y buscar vuelve a ser nodo por nodo, $O(n)$. El AVL se queda en 4 porque mi Insert guarda el regreso con $n \rightarrow \text{left} = \text{InsertRec}(\dots)$ y al subir llama a Balancear(n), que actualiza la altura y rota si el factor llega a 2 o -2. Mi Altura no recorre nada, solo lee el campo $n \rightarrow \text{height}$, por eso es $O(1)$. Si en vez de 15 metiera 1000 numeros en orden, el BST mediria 1000 y el AVL mediria como 10, porque el AVL crece como $\log_2(n)$. Yo lo comprobe con 100 seguidos: mi AVL midio 7.

2. Sobre la rotacion doble: caso LR con 3, 1, 2

Escojo el LR. Meto 3 (raiz, altura 1), luego 1 a la izquierda, luego 2 a la derecha del 1. Mi raiz queda con factor 2 y su hijo izquierdo con factor -1: cargada a la izquierda pero el hijo cargado a la derecha. Si solo aplico RotarDerecha a la raiz, queda 1 arriba, con derecha = 3, y 2 queda como izquierda de 3. El arbol ahora esta cargado del otro lado: el 1 tiene factor -2, sigue desbalanceado. Por eso en mi codigo hago dos pasos: primero $n \rightarrow \text{left} = \text{RotarIzquierda}(n \rightarrow \text{left})$, que sube al 2 arriba del lado izquierdo, y luego return RotarDerecha(n), que sube al 2 como raiz total. Cada rotacion simple solo mueve dos punteros (guardo izq y el subarbol B del medio) y luego actualizo primero al que bajo y despues al que subio. Al final el 2 queda arriba con 1 a la izquierda y 3 a la derecha, altura 2 y balanceado.

3. Sobre el intercambio: trabajo extra que si paga o no

Mi Insert hace trabajo extra en cada nivel al regresar: ActualizarAltura, FactorBalance y hasta dos rotaciones. Sigue siendo $O(\log n)$, pero cada insercion cuesta mas que en un BST normal. Vale la pena cuando busco mucho y los datos vienen ordenados. Ejemplo en un juego: una tabla de puntajes o IDs de jugadores que consulto a cada rato para mostrar el ranking o buscar un jugador por ID. Ahi quiero busqueda $O(\log n)$ siempre, aunque insertar cueste un poco mas. No conviene cuando creo y destruyo cosas a cada rato y casi no busco por orden. Ejemplo: balas o particulas que viven un segundo y solo las recorro todas para moverlas. Ahi es mejor una LinkedList o un arreglo simple: recorrer todo es barato y el AVL solo meteria rotaciones que nadie aprovecha. Si solo necesito buscar exacto sin orden, hasta un HashTable es mejor.

Referencias

VisuAlgo. (s.f.). AVL Tree. <https://visualgo.net/es/bst>

LearnCpp.com. (s.f.). Learn C++. <https://www.learncpp.com/>

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). Introduction to Algorithms (3.a ed.). MIT Press.